

Migrate from PWA Kit to Storefront Next

If you're a PWA Kit developer and are building a Storefront Next storefront, learn about the differences in the technologies and architecture used in PWA Kit and Storefront Next. Storefront Next keeps the same B2C Commerce backend model as PWA Kit (SCAPI, SLAS, and Managed Runtime), but it shifts implementation patterns across the stack. The storefront architecture shifts from a client-heavy, hook-driven architecture in PWA Kit to a server-first architecture built on React Router 7, Vite, and TypeScript in Storefront Next.

Differences in Project Configuration and the Development Workflow

The PWA Kit and Storefront Next are structured, configured, built, and deployed differently. Migrate a custom Webpack-based workflow in PWA Kit to a more convention-driven Vite and TypeScript workflow in Storefront Next.

Project Structure

PWA Kit organizes most app code under `app/` with centralized route definitions and explicit runtime entry files. Storefront Next shifts to `src/`, file-based routes, and framework conventions around root and route modules. The outcome is a more convention-driven layout with clearer

Contents

[Differences in Project Configuration and the Development Workflow](#)

Differences in Data Fetching, State Management, Storage, and Rendering

Differences in API Integration and Routing

Differences in Styling, Components, Static Assets, and SEO



Project Configuration

PWA Kit uses JavaScript configuration with looser runtime validation and flatter structures. Storefront Next introduces typed config definitions, structured namespaces, and an environment-variable override model designed for safer server-client separation. Access patterns also become more context-aware on the server and hook-based in React components.

→ [Project Configuration Differences with PWA Kit](#)

Build Tools

PWA Kit relies on Webpack, Babel, and Jest with multiple build concerns split across separate configs. PWA Kit supports TypeScript optionally through Babel while Storefront Next has TypeScript as a first-class citizen with strict mode enabled. Storefront Next standardizes on Vite, Vitest, and TypeScript-first configuration, which reduces config complexity and speeds up local iteration through faster startup and hot updates. Code splitting and bundle analysis also become more route- and chunk-oriented by default.

→ [Build System Differences with PWA Kit](#)

Development Workflow

PWA Kit development is driven by Webpack dev tooling and JavaScript-centric config. Storefront Next uses Vite and TypeScript for a faster, more integrated workflow with tighter feedback loops and stronger type safety. Operationally, teams typically see faster local startup, faster HMR, and more consistent tooling across dev/build/test tasks.

→ [Development Workflow Differences with PWA Kit](#)



files. Storefront Next leverages Vite for faster development with native ESM, instant HMR, and type-safe configuration. The shift to Vite provides a more modern developer experience with faster feedback loops and better tooling integration.

→ [Deployment Differences with PWA Kit](#)

Differences in Data Fetching, State Management, Storage, and Rendering

The runtime data and app state in PWA Kit is a client-heavy model while it's a server-first model in Storefront Next. Learn about the rendering, caching, and session handling changes in Storefront Next that affect performance, security, and migration strategy.

Data Fetching, Suspense, and Loading States

PWA Kit follows a fetch-on-render model where hooks request data during component render and expose manual loading flags. Storefront Next follows fetch-then-render with route loaders that gather data before component rendering, enabling Suspense and streaming patterns more naturally. This moves data concerns out of presentation components and simplifies loading orchestration.

PWA Kit typically handles loading imperatively with `isLoading` checks and manual fallbacks. Storefront Next uses React Suspense and promise-based route data consumption for declarative loading boundaries. This mechanism enables finer-grained streaming and less repetitive loading logic. To migrate to



Differences with PWA Kit

State Management

While PWA Kit employs a stateless server architecture paired with TanStack Query for client-side caching, Storefront Next uses HTTP cookies for session persistence.

PWA Kit is stateless on the server side – all pages are server-rendered from the perspective of an unauthenticated (guest) shopper. On the client side, PWA Kit relies on TanStack Query (formerly React Query) as its primary mechanism for caching API responses and managing asynchronous state, supplemented by React Context for local state propagation.

Storefront Next uses HTTP cookies to persist authentication and basket state across requests, and React Context for client-side state management. The framework is unopinionated about state management beyond these defaults; developers are free to integrate the state management solution of their choice and customize the project to fit their requirements.

→ [State Management Differences with PWA Kit](#)

Basket Management

PWA Kit and Storefront Next both use SCAPI baskets, but they differ in where basket state is managed and when full basket data is hydrated. Storefront Next emphasizes middleware-driven basket snapshots and loader/action-based updates with lazy hydration by default.

→ [Basket Management Differences with PWA Kit](#)

Rendering

PWA Kit SSR is Express-oriented with React Query hydration and optional island-style hydration controls. Storefront Next uses React Router framework SSR with loader-driven



modules.

→ [Server-Side Rendering \(SSR\)](#)
[Differences with PWA Kit](#)

Storage and Sessions

PWA Kit auth/session data often spans both server and client, including localStorage token usage in common patterns. Storefront Next centralizes auth on the server and uses `httpOnly` cookies as the primary session/token mechanism. This mechanism improves security posture and SSR consistency by reducing client-side token handling.

→ [Storage and Sessions Differences with PWA Kit](#)

Differences in API Integration and Routing

Find out how API calls, server handlers, and route definitions are organized in each storefront architecture. Transition from component-level hooks and centralized routing in PWA Kit to loader and action patterns, and file-based routing.

API Integration

In PWA Kit, API usage is centered on `@salesforce/commerce-sdk-react` hooks and provider-based auth/caching inside components. In Storefront Next, API calls move into route loaders and actions through generated Salesforce B2C Commerce API (SCAPI) clients based on OpenAPI specs. The APIs are provided in the `@salesforce/storefront-next-runtime/scapi` package. Auth is applied through middleware and cache behavior controlled by router revalidation. The result is a thinner client runtime and clearer separation between read and write operations.



PWA Kit server routes are typically centralized as Express handlers in SSR server code. Storefront Next replaces that with file-based resource routes, and loader and action handlers using standard Web Request / Response APIs. This routing approach distributes server behavior across route modules and aligns API endpoints with routing conventions.

→ [Server Routes Differences with PWA Kit](#)

Routing

PWA Kit uses React Router 5 with programmatic route arrays and manual splitting patterns. Storefront Next uses React Router 7 file-based routing with native nested layouts, typed loaders and actions, and automatic route-level splitting. Migrating generally means converting centralized route configuration into route files and moving route data logic into exports.

→ [Routing Differences with PWA Kit](#)

Differences in Styling, Components, Static Assets, and SEO

Check out front-end implementation differences between PWA Kit and Storefront Next, including UI composition, styling systems, asset handling, and metadata patterns. Map Chakra and PropTypes-era patterns in PWA Kit to the TypeScript-first and utility-class approach in Storefront Next.

Styling

PWA Kit styling is largely Chakra CSS-in-JS with prop-based responsive and variant systems. Storefront Next styling centers on Tailwind utility classes, CSS variables, and class-



→ [Styling Differences with PWA Kit](#)

Components

PWA Kit components are typically JavaScript + PropTypes with Chakra primitives and style props. Storefront Next components are TypeScript-first and composed from Radix/shadcn patterns with utility-class styling and CVA-based variants. The main shift is from runtime styling and theme objects to compile-time class composition and typed component contracts.

→ [Component Differences with PWA Kit](#)

Static Assets

PWA Kit uses Webpack-era asset helpers, sprite-based icon patterns, and custom responsive image utilities. Storefront Next adopts Vite-native asset resolution, direct public paths and imports, and component-based responsive image handling with modern preload behavior. Teams typically replace helper-heavy patterns with simpler import and path conventions.

→ [Static Assets Differences with PWA Kit](#)

SEO and Metadata

PWA Kit commonly uses `react-helmet` and supports dynamic API-driven meta tags in current implementations. Storefront Next uses the built-in React Router head/meta capabilities.

→ [SEO and Metadata Differences with PWA Kit](#)

**DID THIS
ARTICLE
SOLVE YOUR
ISSUE?**

Let us know
so we can
improve!

[Share your feedback](#)



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

CON

Trailt
Even
Partn
Blog
Sales
Sales

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#)

[Contact](#) [Use of Cookies](#) [Cookie Preferences](#)  [Your Privacy Choices](#)